



Quick Reference Guide

Docker · CLI · Topics · Partitions · Producer · Consumer

A hands-on learning guide built from real CLI sessions and practical debugging — organised for fast revision.

01	Kafka Fundamentals	Core concepts, broker, ZooKeeper
02	Docker & Container Management	exec, naming, ps commands
03	Topics & Partitions	Create, list, describe, design
04	Producer & Consumer CLI	Send & receive messages
05	Partition Routing & Keys	Keys, routing, per-partition read
06	Command Cheat Sheet	All commands at a glance

01 Kafka Fundamentals

Apache Kafka is a distributed event streaming platform designed to handle high-throughput, fault-tolerant, real-time data pipelines. At its core, Kafka acts as a durable message bus between producers (senders) and consumers (receivers), decoupling them so neither needs to know about the other.

Key Components

Component	Role	Analogy
Broker	The Kafka server — stores & serves messages	Post office
ZooKeeper	Coordinates brokers, manages metadata	Office manager
Topic	A named channel for a category of messages	Mailing list
Partition	Subdivision of a topic for parallelism	Lane on a highway
Producer	Publishes messages to a topic	Sender
Consumer	Reads messages from a topic	Recipient
Offset	Position of a message inside a partition	Page number

How Kafka Fits Together

A Producer sends messages to a Topic. Kafka distributes those messages across the topic's Partitions. One or more Consumers subscribe to the topic and read messages from their assigned partitions. ZooKeeper (or KRaft in newer versions) keeps all brokers in sync.

```
Producer [ Topic: orders ] Consumer Group

partition-0 Consumer A
partition-1 Consumer B
partition-2 Consumer C
```

■ Kafka is NOT HTTP. You cannot open localhost:9092 in a browser. It speaks its own binary TCP protocol. Use a UI tool like Kafka UI, Conduktor, or Redpanda Console if you want a visual interface.

02 Docker & Container Management

The Three Name Types — Know the Difference

Name Type	Example Value	Use it with
Container name	global_kafka_container-kafka-1	docker exec, docker stop, docker logs
Service name	kafka	docker compose exec/stop/logs
Image name	confluentinc/cp-kafka:7.5.0	docker pull, docker build

- Docker auto-generates container names as [project]-[service]-[index]. Always run `docker ps` first to confirm the real container name before using `docker exec`.

Essential Docker Commands

- List ALL running containers (works anywhere)

```
docker ps
```

- List containers for a specific compose project (must be run inside compose folder)

```
docker compose ps
```

- Run a one-off command inside a container

```
docker exec
```

- Run an interactive command (producer/consumer)

```
docker exec -it
```

- View container logs

```
docker logs
```

- Stop a container

```
docker stop
```

docker exec vs docker compose exec

Command	Needs compose folder?	Uses which name?
docker exec	No — works anywhere	Container name
docker compose exec	Yes	Service name

■■ docker compose ps errors with 'no configuration file provided' when run outside the folder containing docker-compose.yml. This is expected — it is NOT a Kafka error.

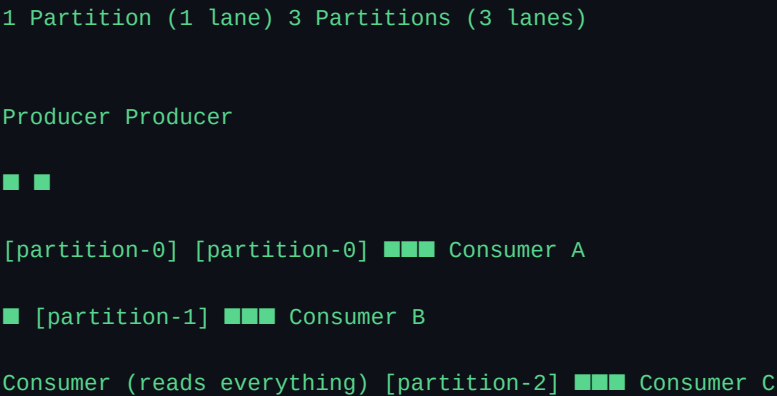
03 Topics & Partitions

What is a Topic?

A topic is a named, append-only log of messages. Think of it as a category or feed — e.g. 'user-signups', 'order-events', 'payment-updates'. Producers write to a topic; consumers read from it. Topics are split into partitions for scalability.

Why Partitions?

Partitions are the unit of parallelism in Kafka. A topic with N partitions can be consumed by up to N consumers simultaneously. Without partitions, a single consumer would have to process all messages sequentially.



	1 Partition	3 Partitions
Max parallel consumers	1	3
Throughput	Baseline	~3× faster
Message order	Global	Per-partition only
Good for	Dev / testing	Production workloads

Topic Management Commands

Create a topic

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--create --topic test-topic \  

```

```
--partitions 3 \  
  
--replication-factor 1
```

List all topics

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--list
```

Describe a topic (see partitions, replicas, leaders)

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--describe --topic test-topic  
  
# Sample output:  
  
Topic: test-topic Partitions: 3 ReplicationFactor: 1  
  
Partition: 0 Leader: 1 Replicas: 1  
  
Partition: 1 Leader: 1 Replicas: 1  
  
Partition: 2 Leader: 1 Replicas: 1
```

Delete a topic

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--delete --topic test-topic
```

Replication Factor — Dev vs Production

replication-factor	Meaning	Use case
1	1 copy — no redundancy	Local dev / learning
2	2 copies — survives 1 failure	Staging environments
3	3 copies — survives 2 failures	Production (recommended)

■■ replication-factor cannot exceed the number of brokers. Since your local setup has 1 broker, use --replication-factor 1.

04 Producer & Consumer CLI

Why -it is Required Here

The `-i` flag keeps STDIN open so you can type messages interactively. The `-t` flag allocates a pseudo-TTY (terminal) so your shell renders properly. Without `-it` the console producer/consumer would exit immediately. One-shot commands like `kafka-topics --create` do not need `-it` because they run, finish, and exit on their own.

Producer — Send Messages to a Topic

```
docker exec -it kafka-console-producer \  
  
--bootstrap-server localhost:9092 \  
  
--topic test-topic  
  
# A '>' prompt appears. Type a message and press Enter to send.  
  
> Hello Kafka  
  
> My first message  
  
> Press Ctrl+C to stop
```

Flag	What it does
<code>docker exec</code>	Execute a command inside a running container
<code>-it</code>	Interactive + TTY — keeps the terminal open for typing
<code>kafka-console-producer</code>	Kafka's built-in CLI tool for sending messages
<code>--bootstrap-server</code>	Address of the Kafka broker (host:port)
<code>--topic</code>	The topic to publish messages to

Consumer — Read Messages from a Topic

```
docker exec -it kafka-console-consumer \  
  
--bootstrap-server localhost:9092 \  
  
--topic test-topic \  
  
--from-beginning
```

Flag	What it does
kafka-console-consumer	Kafka's built-in CLI tool for reading messages
--from-beginning	Read ALL stored messages, not just new ones
(no --from-beginning)	Only receive messages arriving after consumer starts

Understanding --from-beginning

Messages in test-topic:

'Hello' → 'World' → 'Kafka' → [future messages...]

WITHOUT --from-beginning:

Consumer starts HERE (only new messages)

WITH --from-beginning:

Consumer starts HERE



Reads everything from the very first message

- Open the producer in Terminal 1 and the consumer in Terminal 2. Type a message in Terminal 1 and watch it appear in Terminal 2 in real time.

05 Partition Routing & Keys

How Kafka Routes Messages to Partitions

Method	How it works	Order guaranteed?
No key (default)	Round-robin: msg1→p0, msg2→p1, msg3→p2...	No
With key	hash(key) % partitions → same key always same partition	Within partition
Manual (consumer only)	--partition flag on consumer side	Within partition

■■ kafka-console-producer does NOT support a --partition flag. To target a specific partition, use message keys. kafka-console-consumer DOES support --partition.

Producing with Keys

When you send a message with a key, Kafka hashes the key and consistently routes it to the same partition. This is critical for ordered processing of related events (e.g. all events for user-123 must be processed in order).

```
docker exec -it kafka-console-producer \
--bootstrap-server localhost:9092 \
--topic test-topic \
--property 'parse.key=true' \
--property 'key.separator=:' \

# Format: key:message
> user-123:Order placed
> user-123:Payment received # same partition as above
> user-456:Order placed # different partition
```

```
key1 ■■ hash(key1) % 3 = 0 ■■ partition-0
key1 ■■ hash(key1) % 3 = 0 ■■ partition-0 (always same!)
key2 ■■ hash(key2) % 3 = 1 ■■ partition-1
```

```
key3 ■■ hash(key3) % 3 = 2 ■■ partition-2
```

Consuming from a Specific Partition

```
# Read only from partition 0

docker exec -it kafka-console-consumer \

--bootstrap-server localhost:9092 \

--topic test-topic \

--partition 0 \

--from-beginning

# Change --partition 0 to 1 or 2 for other partitions
```

Real-World Use Case

In a production e-commerce system, using the customer ID as the message key ensures all events for a given customer (order placed, payment received, shipped, delivered) land in the same partition and are processed in strict order by a single consumer — while still allowing other customers' events to be processed in parallel.

```
Customer events ■■ Kafka Topic: order-events

■

key=user-001 ■■■■■■■■■■ partition-0 ■■■ Consumer A (processes user-001 in order)

key=user-002 ■■■■■■■■■■ partition-1 ■■■ Consumer B (processes user-002 in order)

key=user-003 ■■■■■■■■■■ partition-2 ■■■ Consumer C (processes user-003 in order)
```

06 Command Cheat Sheet

Replace with your actual container name (e.g. global_kafka_container-kafka-1). Run `docker ps` to confirm the exact name.

Docker — Container Inspection

- List all running containers

```
docker ps
```

- List compose project containers

```
docker compose ps # run in compose folder
```

- View container logs

```
docker logs
```

Topic Management

- Create topic

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--create --topic --partitions 3 --replication-factor 1
```

- List topics

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 --list
```

- Describe topic

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--describe --topic
```

- Delete topic

```
docker exec kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--delete --topic
```

Producer

■ Basic producer (no key)

```
docker exec -it kafka-console-producer \  
  
--bootstrap-server localhost:9092 --topic
```

■ Producer with key

```
docker exec -it kafka-console-producer \  
  
--bootstrap-server localhost:9092 --topic \  
  
--property 'parse.key=true' \  
  
--property 'key.separator=:'
```

Consumer

■ Consume all (from start)

```
docker exec -it kafka-console-consumer \  
  
--bootstrap-server localhost:9092 --topic \  
  
--from-beginning
```

■ Consume new messages only

```
docker exec -it kafka-console-consumer \  
  
--bootstrap-server localhost:9092 --topic
```

■ Consume specific partition

```
docker exec -it kafka-console-consumer \  
  
--bootstrap-server localhost:9092 --topic \  
  
--partition 0 --from-beginning
```

Quick Decision Guide

I want to...	Use this
Run a command in a container from anywhere	docker exec
Run interactively (type messages)	docker exec -it
Use service name shorthand	docker compose exec (from compose folder)
Read all past + future messages	--from-beginning on consumer
Keep related messages ordered	Use message keys on producer

Read one partition at a time	--partition N on consumer
------------------------------	---------------------------

Keep experimenting — spin up a producer in one terminal and a consumer in another. That real-time feedback loop is the fastest way to solidify these concepts.

07 Python Producer & Consumer

What is confluent_kafka?

`confluent_kafka` is the official Python client library for Apache Kafka, maintained by Confluent. It wraps the high-performance `librdkafka` C library, making it much faster than the pure-Python `kafka-python` alternative. It gives you `Producer`, `Consumer`, and `AdminClient` classes to interact with Kafka programmatically — no CLI needed.

```
pip install confluent-kafka
```

Producer Code — Full Breakdown

```
from confluent_kafka import Producer

import json, time

producer = Producer({'bootstrap.servers': 'localhost:9092'})
```

Creates a Kafka producer connected to your broker. This is the Python equivalent of `--bootstrap-server localhost:9092` in the CLI.

```
def delivery_report(err, msg):

    if err:

        print(f'Delivery failed: {err}')

    else:

        print(f'Delivered: {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}')
```

A callback function — Kafka calls this automatically after each message is either successfully stored or fails. It tells you exactly which topic, partition, and offset the message landed at. You never call this yourself — Kafka triggers it internally.

```
for i in range(10):

    event = {

        'user_id': f'user_{i}',

        'action': 'page_view',

        'timestamp': int(time.time()),
```

```

'page': f'/product/{i * 3}' # 0, 3, 6, 9...
}

producer.produce(

topic = 'user_events',

value = json.dumps(event), # serialize dict to string

key = event['user_id'], # routing key

callback = delivery_report

)

producer.poll(0) # trigger event loop non-blocking

producer.flush() # BLOCK until all messages are confirmed

print('Done!')
```

Method / Config	What it does
<code>producer.produce()</code>	Queues the message — does NOT send immediately
<code>producer.poll(0)</code>	Triggers Kafka's internal event loop, fires callbacks
<code>producer.flush()</code>	Blocks until every queued message is delivered — safety net
<code>key=</code>	Determines which partition the message routes to
<code>value=</code>	The actual message body — must be bytes or string
<code>callback=</code>	Function called after Kafka confirms delivery or failure

Consumer Code — Full Breakdown

```

from confluent_kafka import Consumer

import json

consumer = Consumer({

'bootstrap.servers': 'localhost:9092',

'group.id': 'my_consumer_group', # tracks read position per group

'auto.offset.reset': 'earliest' # start from beginning if no saved offset

})
```

```
consumer.subscribe(['user_events'])
```

Config	Meaning	CLI Equivalent
group.id	Groups consumers together — Kafka tracks offsets per group	N/A (CLI is stateless)
auto.offset.reset: earliest	Read from offset 0 if no saved position	--from-beginning
auto.offset.reset: latest	Only read new messages arriving after start	(default, no flag)

```
try:

while True:

msg = consumer.poll(1.0) # wait up to 1 sec for a message

if msg is None:

continue # no message yet, keep waiting

if msg.error():

print(f'Error: {msg.error()}')

continue

event = json.loads(msg.value().decode('utf-8'))

print(f'[Partition {msg.partition()}, Offset {msg.offset()}]')

print(f" User: {event['user_id']} visited {event['page']}")

except KeyboardInterrupt:

pass

finally:

consumer.close() # commits offsets + leaves consumer group cleanly
```

Method	What it does
consumer.poll(1.0)	Waits up to 1 second for a message, returns None if nothing arrives
msg.value()	Returns raw bytes — must .decode('utf-8') then json.loads()
msg.partition()	Which partition this message came from
msg.offset()	The sequential position of this message in the partition
consumer.close()	Commits current offset + notifies Kafka this consumer left the group

Understanding the Output

All 10 messages landed on partition 0 even though `user_events` had 3 partitions. This is because Kafka hashes the key and applies modulo — with keys `user_0` through `user_9`, all hashes happened to resolve to partition 0. With thousands of diverse keys in production, distribution would be much more even.

```
# Producer output — all on partition 0:

Delivered: user_events [0] @ offset 0

Delivered: user_events [0] @ offset 1

... # hash('user_0..9') % 3 = 0 for all


# Consumer output — reads in exact insertion order:

[Partition 0, Offset 0]

User: user_0 visited /product/0

[Partition 0, Offset 1]

User: user_1 visited /product/3 # i*3 pattern: 0,3,6,9,12...
```

- Order is guaranteed WITHIN a partition. Since all messages went to partition 0, they were consumed in perfect insertion order (offset 0 → 9).

08 Controlling Topics from Python — AdminClient

The Auto-Creation Problem

When your producer writes to a topic that doesn't exist, Kafka auto-creates it using its default settings — typically 1 partition. This is why all messages in the previous chapter landed on partition 0: `user_events` was auto-created with only 1 partition, so there was nowhere else to go.

```
# Verify partition count of any topic:

docker exec kafka-topics \

--bootstrap-server localhost:9092 \

--describe --topic user_events


# Likely output if auto-created:

Topic: user_events Partitions: 1 ReplicationFactor: 1

Partition: 0 Leader: 1 Replicas: 1 # only 1 partition!
```

3 Ways to Fix Partition Count

Option	Command / Approach	Best for
Delete & recreate	<code>kafka-topics --delete</code> then <code>--create</code> with <code>--partitions 3</code>	Dev / clean slate
Alter existing topic	<code>kafka-topics --alter --partitions 3</code>	Keep existing messages
AdminClient in Python	Use <code>confluent_kafka.admin</code> before producing	Production / real projects

■■ You can only INCREASE partition count, never decrease. Existing messages stay in their original partitions — only new messages spread across all partitions.

Option 1 — Delete & Recreate via CLI

The cleanest reset for local dev. Deletes all existing messages and recreates the topic with the exact partition count you want.

Step 1 — Delete the auto-created topic

```
docker exec global_kafka_container-kafka-1 kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--delete --topic user_events
```

Step 2 — Recreate with 3 partitions

```
docker exec global_kafka_container-kafka-1 kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--create --topic user_events \  
  
--partitions 3 \  
  
--replication-factor 1
```

Verify the result

```
docker exec global_kafka_container-kafka-1 kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--describe --topic user_events  
  
# Expected output after fix:  
  
Topic: user_events Partitions: 3 ReplicationFactor: 1  
  
Partition: 0 Leader: 1 Replicas: 1  
  
Partition: 1 Leader: 1 Replicas: 1  
  
Partition: 2 Leader: 1 Replicas: 1
```

■ After recreating with 3 partitions, re-run `producer.py`. Messages will now spread across partitions — you will see `[0]`, `[1]`, and `[2]` in the delivery output.

Option 2 — Alter Existing Topic via CLI

Use this when you want to increase partitions without losing existing messages. Note that only new messages will spread to the added partitions.

```
docker exec global_kafka_container-kafka-1 kafka-topics \  
  
--bootstrap-server localhost:9092 \  
  
--alter --topic user_events \  
  
--partitions 3
```

AdminClient — Create Topic from Python

The cleanest approach is to create the topic programmatically inside your producer script. This makes the producer fully self-contained — no manual CLI setup needed before running it.

```
from confluent_kafka.admin import AdminClient, NewTopic

admin = AdminClient({'bootstrap.servers': 'localhost:9092'})

new_topic = NewTopic(
    topic='user_events',
    num_partitions=3,
    replication_factor=1
)

result = admin.create_topics([new_topic])

for topic, future in result.items():
    try:
        future.result()

        print(f'Topic "{topic}" created with 3 partitions')

    except Exception as e:
        # Topic already exists – that is fine, just skip
        print(f'Topic "{topic}" already exists or error: {e}')
```

■ The except block is critical. On the second run, Kafka throws an error because the topic already exists. Catching the exception means the script continues normally instead of crashing.

Complete Self-Contained Producer (Recommended Pattern)

Combine AdminClient + Producer in one file so the topic is always created with the right settings before any messages are sent:

```
from confluent_kafka import Producer

from confluent_kafka.admin import AdminClient, NewTopic

import json, time
```

What Happens on Each Run

Scenario	What the code does
----------	--------------------

First run — topic does not exist	Creates user_events with 3 partitions, then produces
Second run — topic already exists	Exception caught silently, continues producing normally
Topic exists with wrong partitions	Does NOT fix it — delete and recreate manually via CLI

Docker + Python — How They Connect

Your Python scripts and the Kafka CLI both talk to the same broker. Docker maps the container's internal port 9092 to your Mac's localhost:9092, so any code using localhost:9092 reaches the same Kafka instance.

```

Your Mac

■■■ Docker: global_kafka_container-kafka-1

■ ■■■ Kafka Broker (internal port 9092)

■ ▲ ▲

■ docker-compose.yml: ■

■ ports: '9092:9092' ■

■ ■ ■

■■■ producer.py consumer.py

localhost:9092 localhost:9092

(same broker!) (same broker!)
```

This is confirmed by your docker-compose.yml having ports: '9092:9092' which maps Mac localhost:9092 directly into the container. Kafka auto-creates topics when a producer writes to a non-existent one (auto.create.topics.enable=true by default).

You now have the full picture — from Docker container naming to Python producers managing their own topics. The next step is running multiple consumers in a group and watching Kafka balance partitions between them automatically.

09 Real-World ML Pipeline — 3 Scripts

Architecture Overview

Three Python scripts form a complete real-time ML pipeline. Each is a separate process connected through Kafka topics. They can be scaled, restarted, or replaced independently.

```
producer.py transformer.py consumer.py

make_event() ■■■ raw-events ■■■ features ■■■ ml-features ■■■ predict()

every 0.2s [topic] per user [topic] discount>70%
```

Bugs in Original Code — Learn From These

Bug	File	Problem	Fix
1	consumer.py	Uses session_duration_sec, is_mobile, on_checkout — transformer never produces these → KeyError	Add missing fields to transformer
2	producer.py	bootstrap.server missing 's' — producer silently fails to connect	bootstrap.servers
3	transformer.py	Produces to 'features' but consumer reads 'ml-features' — messages never arrive	Change to ml-features

Corrected transformer.py Features Dict

```
features = {

    'user_id': user_id,

    'page_views': state['page_views'],

    'unique_pages': len(state['pages_seen']),

    'session_duration_sec': event['timestamp'] - state['session_start'],

    'is_mobile': 1 if state['last_device'] == 'mobile' else 0,

    'on_checkout': 1 if event['page'] == '/checkout' else 0

}

producer.produce('ml-features', key=user_id, value=json.dumps(features))
```

■ Start order: transformer.py first, then consumer.py, then producer.py — consumers must be ready before events flow.

ML Inference — Logistic Regression Explained

```
WEIGHTS = np.array([0.3, 0.5, 0.1, 0.4, -0.2])

BIAS = -2.0

def predict_purchase_prob(f):

    x = np.array([f['page_views'], f['unique_pages'],
                  f['session_duration_sec'], f['is_mobile'], f['on_checkout']])

    logit = x @ WEIGHTS + BIAS # dot product = weighted sum

    return float(1 / (1 + np.exp(-logit))) # sigmoid → 0.0 to 1.0
```

Part	What it does
<code>x @ WEIGHTS</code>	Dot product — each feature multiplied by weight, summed
<code>+ BIAS</code>	Shifts the decision boundary
<code>sigmoid</code>	Converts raw score to probability 0-1
<code>prob > 0.7</code>	Business rule: 70% confidence to show discount

10 Production Producer Configuration

Every Parameter Explained

```
producer = Producer({  
  
    'bootstrap.servers': 'broker1:9092,broker2:9092,broker3:9092',  
  
    'acks': 'all',  
  
    'retries': 5,  
  
    'retry.backoff.ms': 500,  
  
    'linger.ms': 5,  
  
    'batch.size': 65536,  
  
    'compression.type': 'lz4',  
  
    'enable.idempotence': True,  
  
})
```

Parameter	Dev Default	Production Value	Effect
bootstrap.servers	localhost:9092	broker1,2,3	Cluster fault tolerance
acks	1	all	No data loss — all replicas confirm
retries	0	5	Survive transient network failures
retry.backoff.ms	100ms	500ms	Wait between retry attempts
linger.ms	0	5ms	Batch messages for efficiency
batch.size	16KB	64KB	Larger batches = fewer network calls
compression.type	none	lz4	60-70% bandwidth reduction
enable.idempotence	False	True	No duplicates on retry

acks=all safety guarantee:

acks=1: produce → leader ✓ → leader crashes → DATA LOST

acks=all: produce → leader ✓ → replica1 ✓ → replica2 ✓ → safe

idempotence prevents duplicates on retry:

Without: `produce(seq=1)` → ACK lost → retry → DUPLICATE in topic

With: `produce(seq=1)` → ACK lost → retry → Kafka: already have seq=1 → ignored

`linger.ms` batching:

`linger=0`: `msg1`→send, `msg2`→send, `msg3`→send = 3 network calls

`linger=5`: [`msg1`,`msg2`,`msg3`] wait 5ms → send = 1 network call

11 Production Consumer Config & Manual Commit

What is an Offset Commit?

Committing saves your reading position. On restart, the consumer asks Kafka where it left off and resumes from the last committed offset. Wrong commits = lost or duplicate messages.

```
[msg1][msg2][msg3][msg4][msg5]

committed=2

Crash → restart → Kafka: start from offset 3
```

Config Parameters

Parameter	Value	Meaning
enable.auto.commit	False	YOU control when offset is committed
session.timeout.ms	30000	30s without heartbeat → partition reassigned
max.poll.interval.ms	300000	5min between poll() calls → partition reassigned
auto.offset.reset	earliest	Start from beginning if no saved offset

```
# Auto-commit OFF – safe for ML:

t=0s poll() → offset=5

t=1s inference CRASHES → no commit

t=2s restart → Kafka redelivers offset=5


# Auto-commit ON – dangerous:

t=0s poll() → offset=5

t=5s AUTO-COMMIT → offset=5 marked done

t=6s inference CRASHES

t=7s restart → Kafka starts at offset=6 → msg5 PERMANENTLY LOST


# Always commit LAST:
```

```
run_model_inference(msg) # 1. process  
  
save_result(result) # 2. persist  
  
consumer.commit(msg) # 3. only now commit
```

12 Dead Letter Queue Pattern

Why DLQ Exists

One broken message retries forever without a DLQ, blocking all subsequent messages. DLQ isolates bad messages so the pipeline keeps moving.

```
def process_with_dlq(msg):  
  
    try:  
  
        event = json.loads(msg.value())  
  
        result = run_model_inference(event)  
  
        consumer.commit(msg) # SUCCESS  
  
    except json.JSONDecodeError as e:  
  
        dlq.produce('ml-features-dlq', # BAD MESSAGE → DLQ  
value=msg.value(),  
headers={'error': str(e)})  
  
        consumer.commit(msg) # skip permanently  
  
    except Exception as e:  
  
        print(f'Will retry: {e}')  
  
        raise # TRANSIENT → no commit → retry
```

Exception	Retry helps?	Action	Commit?
JSONDecodeError	No — always broken	Ship to DLQ	Yes — skip
ConnectionError	Yes — temporary	raise	No — retry
ModelTimeoutError	Yes — temporary	raise	No — retry
Success	N/A	normal flow	Yes

■ Headers carry error metadata alongside the broken message so you know exactly why it was rejected when inspecting the DLQ later.

13 Monitoring with Prometheus

4 Metric Types

Type	Direction	Use for	Example
Counter	Only up	Events that happened	messages_processed_total
Histogram	Distribution	Latency, duration	inference_duration_seconds
Gauge	Up and down	Current state	consumer_lag_messages
Summary	Distribution	Client-side quantiles	Less common, prefer Histogram

```

from prometheus_client import Counter, Histogram, Gauge, start_http_server

start_http_server(8000) # expose metrics at :8000/metrics

PROCESSED = Counter('messages_processed_total', 'Messages processed')

LATENCY = Histogram('inference_duration_seconds', 'Inference time')

LAG = Gauge('consumer_lag_messages', 'Lag', ['partition'])

DLQ_COUNT = Counter('dlq_messages_total', 'DLQ messages')

# In your loop:

with LATENCY.time(): # auto-measures time inside block
    process(msg)

    PROCESSED.inc() # +1 per message

    LAG.labels(partition=str(msg.partition())).set(current_lag)

# In DLQ handler:

DLQ_COUNT.inc()

```

Alert	Metric	Condition
Consumer stopped	messages_processed_total	rate == 0 for 5min
Inference too slow	inference_duration_seconds	p99 > 2s
Pipeline behind	consumer_lag_messages	lag > 1000

Bad data spike	dlq_messages_total	rate > 10/min
----------------	--------------------	---------------

14 Kafka UI & Docker Compose

Common Docker Compose Mistakes

Mistake	Fix
kafka-ui at root level (not under services)	Indent under services: same level as kafka and zookeeper
BOOTSTRAPSERVERS: localhost:9092	Use service name: kafka:29092 — localhost inside container = itself
BOOTSTRAPSERVERS: kafka:9092	Use internal port 29092, not external 9092
Single listener PLAINTEXT://localhost:9092	Use dual listeners: internal (29092) + external (9092)

Two listeners explained:

```
KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.0:9092
```

```
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
```

kafka:29092 ← containers talk to each other via service name

localhost:9092 ← your Mac Python scripts use this (Docker port mapped)

15 Next Steps — 6 Knowledge Gaps

Your foundation is solid — from CLI basics to production ML pipelines with DLQ, monitoring, and idempotent producers. These 6 areas are the natural next layer.

Gap 1 — Consumer Groups and Partition Rebalancing

What happens when consumers join or leave a group. Max useful consumers equals number of partitions.

```
# 1 consumer, 3 partitions:

Consumer A → [p0, p1, p2]

# Add Consumer B → REBALANCE:

Consumer A → [p0, p1]

Consumer B → [p2]

# Add Consumer C → perfect parallelism:

Consumer A → [p0] Consumer B → [p1] Consumer C → [p2]

# Add Consumer D → sits idle (more consumers than partitions):

Consumer D → []

RULE: max useful consumers = number of partitions
```

Gap 2 — Offset Seek and Replay

Reprocess historical messages after fixing a bug in your ML model.

```
from confluent_kafka import TopicPartition

# Replay from beginning (after bug fix):

consumer.seek(TopicPartition('ml-features', partition=0, offset=0))

# Commit specific offsets:

consumer.commit(offsets=[

    TopicPartition('ml-features', 0, offset=142),
```

```
TopicPartition('ml-features', 1, offset=89),

])

# Sync vs async commit:

consumer.commit(msg) # sync – waits for ACK

consumer.commit(msg, asynchronous=True) # async – faster, no confirmation
```

Gap 3 — Exactly Once Semantics

Atomic consume + produce + commit. Either all happen or none do. Critical for payments and orders.

```
At-most-once: 0 or 1 times (auto-commit – can lose messages)

At-least-once: 1+ times (manual commit – can duplicate)

Exactly-once: exactly 1 time (transactions – hardest, safest)

producer = Producer({'transactional.id': 'ml-pipeline-v1', 'enable.idempotence':
True})

producer.init_transactions()

try:

producer.begin_transaction()

producer.produce('ml-results', value=json.dumps(result))

producer.send_offsets_to_transaction(...)

producer.commit_transaction() # atomic – all or nothing

except:

producer.abort_transaction() # rolls everything back
```

Gap 4 — Log Retention and Compaction

How long Kafka keeps messages and log compaction which keeps only the latest message per key.

```
# Time retention (default 7 days):

log.retention.hours=168 # messages deleted after 7 days

# Log compaction – only latest per key:

Before: [user_1:views=1][user_1:views=2][user_1:views=3]
```

```
After: [user_1:views=3] # only latest kept

# Enable compaction:

docker exec kafka-configs \

--bootstrap-server localhost:9092 \

--entity-type topics --entity-name ml-features \

--alter --add-config cleanup.policy=compact
```

Gap 5 — Schema Registry and Avro

Enforce message structure centrally so a renamed field in one service does not silently crash another.

```
# Without Schema Registry – silent crashes:

Producer sends: {'user_id': 'u1', 'page_views': 5}

Producer changes: {'user_id': 'u1', 'pageviews': 5} ← renamed!

Consumer crashes: KeyError: 'page_views' ← no warning


# With Schema Registry – breaking changes caught early:

schema_str = '{"type":"record","name":"MLFeatures","fields":['

{"name":"user_id","type":"string"},

{"name":"page_views","type":"int"}

]}'

# Producer validates every message against registered schema

# Wrong type or missing field → rejected BEFORE sending
```

Gap 6 — Faust Stream Processing

Replace your manual transformer.py with a fault-tolerant framework. State stored in RocksDB and survives restarts.

```
# Your current approach – state lost on crash:

user_state = defaultdict(lambda: {'page_views': 0})

# Restart → all counters reset to 0


# Faust – persistent state, survives restarts:
```

```

import faust

app = faust.App('ml-pipeline', broker='kafka://localhost:9092')

user_table = app.Table('user_state', default=dict) # RocksDB backed

@app.agent(app.topic('raw-events'))

async def process(events):

    async for event in events:

        state = user_table[event['user_id']]

        state['page_views'] = state.get('page_views', 0) + 1

        user_table[event['user_id']] = state # persisted automatically

```

Week	Focus	Why
Week 1	Consumer Groups + Offset Seek	Most immediately useful — rebalancing and replay
Week 2	Log Retention + Schema Registry	Data lifecycle and type safety across services
Week 3	Exactly-Once Semantics	Production reliability for payments and orders
Week 4	Faust Stream Processing	Replace manual transformer with fault-tolerant framework

You have gone from zero to a production-grade Kafka ML pipeline in one session. The gaps above are not beginner topics — they are what separates a working prototype from a system you can confidently deploy in production.